

UNIwersytet Ekonomiczny w Katowicach
Kierunek Informatyka Analiza danych

Adam Kowalczyk

Projekt: Analiza danych jakościowych i Text Mining

Analiza artykułów sportowych

KATOWICE 2025

Spis treści

1. Przygotowanie korpusu	3
2. Chmury słów	4
3. Dzielenie tekstów na grupy i tematy	7
4. Klasyfikacja	14
5. Polaryzacja nastroju.....	17

Projekt ma na celu porównać i wyodrębnić różnice i podobieństwa pomiędzy zbiorami artykułów sportowych z trzech różnych dyscyplin: piłki nożnej, koszykówki i futbolu amerykańskiego.

1. Przygotowanie korpusu

Wpierw należało zebrać kilkadziesiąt artykułów sportowych z zaufanych źródeł sportowych- głównie poświęconych wpisom o głównych ligach sportowych dla danej dyscypliny. Dane zostały wgrane z przygotowanych plików tekstowych, w których każda linia reprezentuje osobny dokument

Następnie wdrożony został proces czyszczenia tekstów w metodzie "clean_tokenize" gdzie usunięte zostały wszystkie znaki interpunkcyjne oraz liczby. Duże litery zamieniono na małe. Do tokenizacji tekstu użyta została biblioteka nltk.

Wprowadzone zostały też "stop words"- często występujące słowa które utrudniałyby analizę i nie niosą istotnego znaczenia. Zastosowano zestaw tych słów z wcześniej wspomnianej biblioteki nltk oraz dodatkowe słowa własne które były dodawane wraz z pracą na dalszych częściach projektu. Między innymi z uwagi na źródło tekstów pochodzące ze stron internetowych i blogów, usunięto słowa takie jak "image", "read", "share" i tym podobne. Dokonano również lematyzacji aby wprowadzić wyrazy do formy podstawowej np. problematyczne i częste występujące słowo "players" zostało zrównane do "player" co usunęło niepotrzebne powtórzenia.

Wynikiem tych operacji był 3 zbiory danych które następnie zostały połączone do jednego jako "corpus". Zredukowano słownictwo do tokenów, które wstąpiły przynajmniej 5 razy- aby zmniejszyć przepełniony korpus.

```
min_freq = 5 #minimalna ilość powtórzeń słowa
vocabulary = {token for token, freq in token_freq.items() if freq >= min_freq}

corpus_final = [[token for token in doc if token in vocabulary] for doc in corpus]

print(f"Liczba dokumentów (linii) : {len(corpus_final)}")
print(f"\nŚrednia długość dokumentów (linii po czyszczeniu): {sum(len(doc) for doc in corpus_final) // len(corpus_final)} słów")
print(f"\nLiczba unikalnych termów po redukcji: {len(vocabulary)}")
```

Liczba dokumentów (linii) : 5614

Średnia długość dokumentów (linii po czyszczeniu): 11 słów

Liczba unikalnych termów po redukcji: 2888

+ Kod + Tekst

Na zakończenie tej wstępnej analizy, obliczona została łączna liczba linii dokumentów, średnią długość dokumentu (linii) i podano liczbę unikalnych tokenów w końcowym słowniku

Rysunek 1. Analiza dokumentów i tokenów w badanych tekstach.

```
my_stopwords = {
    "me", "my", "myself", "we", "our", "ours", "ourselves", "you", "your", "yours", "yourself", "yourselves",
    "he", "him", "his", "himself", "she", "her", "hers", "herself", "it", "its", "itself", "they", "them",
    "their", "theirs", "themselves", "what", "which", "who", "whom", "this", "that", "these", "those", "Aaron",
    "am", "is", "are", "was", "were", "be", "been", "being", "have", "has", "had", "having", "do", "does",
    "did", "doing", "a", "an", "the", "and", "but", "if", "or", "because", "as", "until", "while", "of", "overall",
    "at", "by", "for", "with", "about", "against", "between", "into", "through", "during", "before", "after", "No.", "Michael",
    "above", "below", "to", "from", "up", "down", "in", "out", "on", "off", "over", "under", "again", "further", "er", "got",
    "then", "once", "here", "there", "when", "where", "why", "how", "all", "any", "both", "each", "few", "video", "read", "image",
    "more", "most", "other", "some", "such", "no", "nor", "not", "only", "own", "same", "so", "than", "too", "even", "share", "played",
    "very", "s", "t", "can", "will", "just", "don", "should", "now", "see", "article", "also", "said", "would", "say", "get", "like" #,
}
stop_words = set(stopwords.words('english')).union(my_stopwords)

def clean_tokenize(text):
    text = text.lower()
    text = re.sub(r'\d+', '', text) #usunięcie liczb
    text = re.sub(rf"[{re.escape(string.punctuation)}]", '', text) #interpunkcja- !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
    tokens = word_tokenize(text)
    cleaned_tokens = []
    for token in tokens:
        if token not in stop_words and len(token) > 2:
            lemma = lemmatizer.lemmatize(token) #lematyzacja
            cleaned_tokens.append(lemma)
    return cleaned_tokens
```

Rysunek 2. Czyszczenie tekstów.

2. Chmury słów

Kolejnym etapem analiza słów występujących we wszystkich trzech plikach tekstowych. W celu uzyskania danych zbiorów słów, wpieryw utworzono kolejno listy tokenów z poprzednich list dokumentów. Aby ustalić unikalne terminy dla kolejnych dyscyplin sportowych, należało policzyć wystąpienia dzięki strukturze "Counter". Następnie wyodrębnione zostały unikalne słowa występujące tylko dla jednej kategorii sportu. Biblioteka "wordCloud" umożliwiła wizualizację wyników dla najczęściej występujących unikalnych słów.

Na tym etapie analizy wygenerowane zostały cztery chmury słów:

- Słowa unikalne dla piłki nożnej,
- Słowa unikalne dla koszykówki,
- Słowa unikalne dla futbolu amerykańskiego,
- Słowa wspólne dla tych trzech sportów

```

european_f = [token for doc in european_clean for token in doc]
basketball_f = [token for doc in basketball_clean for token in doc]
american_f = [token for doc in american_clean for token in doc]

counter_european = Counter(european_f)
counter_basketball = Counter(basketball_f)
counter_american = Counter(american_f)

common_words = set(counter_european.keys()) & set(counter_american.keys()) & set(counter_basketball.keys())
common_freq = {word: counter_european[word] + counter_american[word] + counter_basketball[word] for word in common_words}

unique_european = set(counter_european.keys()) - set(counter_american.keys()) - set(counter_basketball.keys())
unique_basketball = set(counter_basketball.keys()) - set(counter_european.keys()) - set(counter_american.keys())
unique_american = set(counter_american.keys()) - set(counter_european.keys()) - set(counter_basketball.keys())

unique_european_freq = {word: counter_european[word] for word in unique_european}
unique_basketball_freq = {word: counter_basketball[word] for word in unique_basketball}
unique_american_freq = {word: counter_american[word] for word in unique_american}

def generate_wordcloud(freq_dict, title):
    wordcloud = WordCloud(width=800, height=400,
                           background_color='white',
                           colormap='viridis').generate_from_frequencies(freq_dict)
    plt.figure(figsize=(10, 5))
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.axis('off')
    plt.title(title, pad=20)
    plt.show()

```

Rysunek 3. Generowanie chmur słów.

Unikalne terminy: European Football



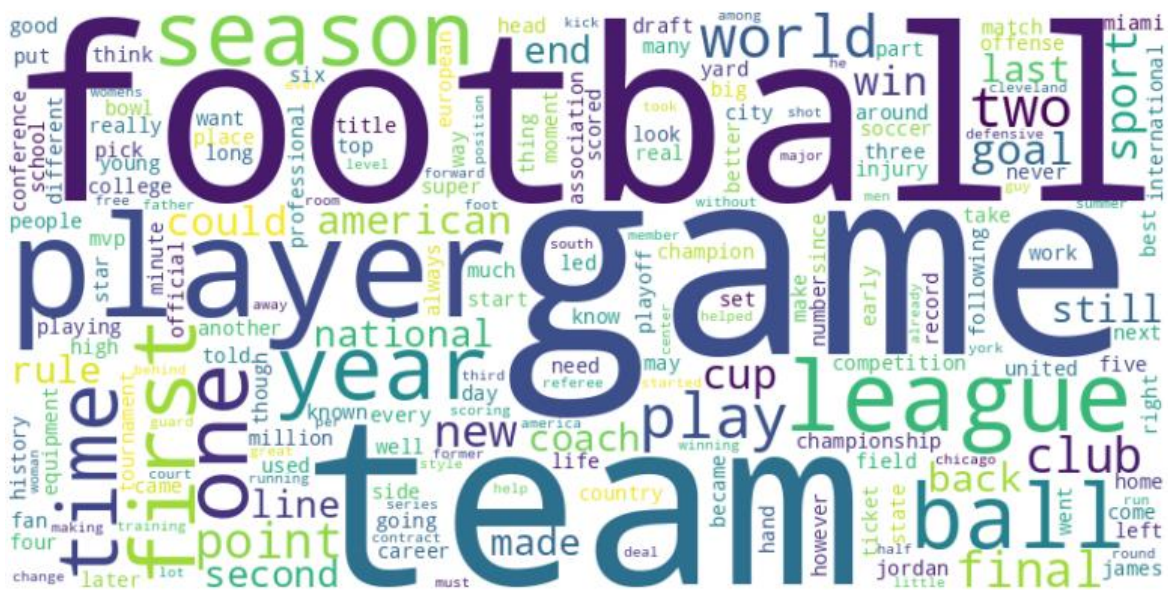
Rysunek 4. Chmura słów przedstawiająca słowa unikalne dla artykułów o piłce nożnej.

Rysunek 5. Chmura słów przedstawiające słowa unikalne dla artykułów o futbolu amerykańskim.

[illegible]

Rysunek 6. Chmura słów przedstawiające słowa unikalne dla artykułów o koszykówce.

Wspólne temy



Rysunek 7. Chmura słów przedstawiające słowa wspólne dla artykułów o wszystkich trzech sportach.

Jak można zaważyć na podstawie wizualizacji chmur, unikalne słowa reprezentują między innymi:

- nazwę głównej ligi sportu,
- nazwiska słynnych sportowców dla danej dziedziny,
- specjalne pozycje, zasady, rodzaje zawodników,
- nazwy drużyn sportowych,
- krajów najbardziej słynnych dla danej dziedziny,
- wyposażenie sportowe

Wspólne słowa za to zawierają ogólne wyrażenia używane przy relacjach z meczów czy elementów łączących dane sporty.

3. Dzielenie tekstów na grupy i tematy

Pierwszym celem tej analizy było automatyczne pogrupowanie dokumentów tekstowych dyscyplin sportowych i sprawdzenie jak algorytm poradzi sobie z podzieleniem na kategorie tekstów o tak dużej liczbie znaków i podobieństwach w używanym słownictwie.

Po oczyszczeniu tekstów, przyszedłszy do wektoryzacji. Użyto "TfidfVecotrizer" z ograniczeniem do 1000 najczęstszych słów. Zastosowano metodę "TF-IDF" aby nadać wyższe wagi dla charakterystycznych wyrazów w dokumentach a niższe powszechnym słowom.

Zastosowana została analiza głównych składowych (PCA) aby móc zwizualizować dane. Do pogrupowania tekstów użyliśmy algorytm "KMeans"- dzieląc je na 2 grupy aby zobaczyć jak poradzi sobie z zakwalifikowaniem słów do dwóch dyscyplin. Na poniższych wykresach os

PCA 1 i PCA 2 wyjaśniają części wariancji w danych. Lewy wykres przedstawia przewidywania KMeans a drugi podział dokumentów zgodny z własnym oznaczeniem (niebieski- pierwszy zestawiany sport, czerwony- drugi zestawiany sport). Na końcu przeprowadzono grupowanie dla wszystkich 3 zbiorów artykułów razem na 3 grupy.

Drugim celem tej analizy było nadanie tematów dla plików z artykułami i identyfikacja charakterystycznych dla nich słów, w celu sprawdzenia, czy dane zawierają ukrytą strukturę tematyczną odpowiadającą badanym dyscyplinom sportowym. Do tego zadania wykorzystano metodę modelowania tematów a konkretnie algorytm LDA.

Dzięki wcześniejszym przetworzeniu dokumentów do postaci wektorów, algorytm przypisuje najważniejsze słowa do każdego z wykrytych tematów. W tej analizie początkowo założono po dwa tematy- aby wpierv porównać po dwie dyscypliny sportowe, a na końcu założono trzy tematy dla wszystkich kategorii sportowych.

```
plt.subplot(1, 2, 1)
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=kmeans_labels, cmap='coolwarm', alpha=0.6)
plt.title("Grupowanie KMeans (2 klastry)")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")

plt.subplot(1, 2, 2)
colors = ['blue' if label == 'european' else 'red' for label in true_labels]
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=colors, alpha=0.6)
plt.title("Prawdziwe kategorie dokumentów")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")

plt.tight_layout()
plt.show()

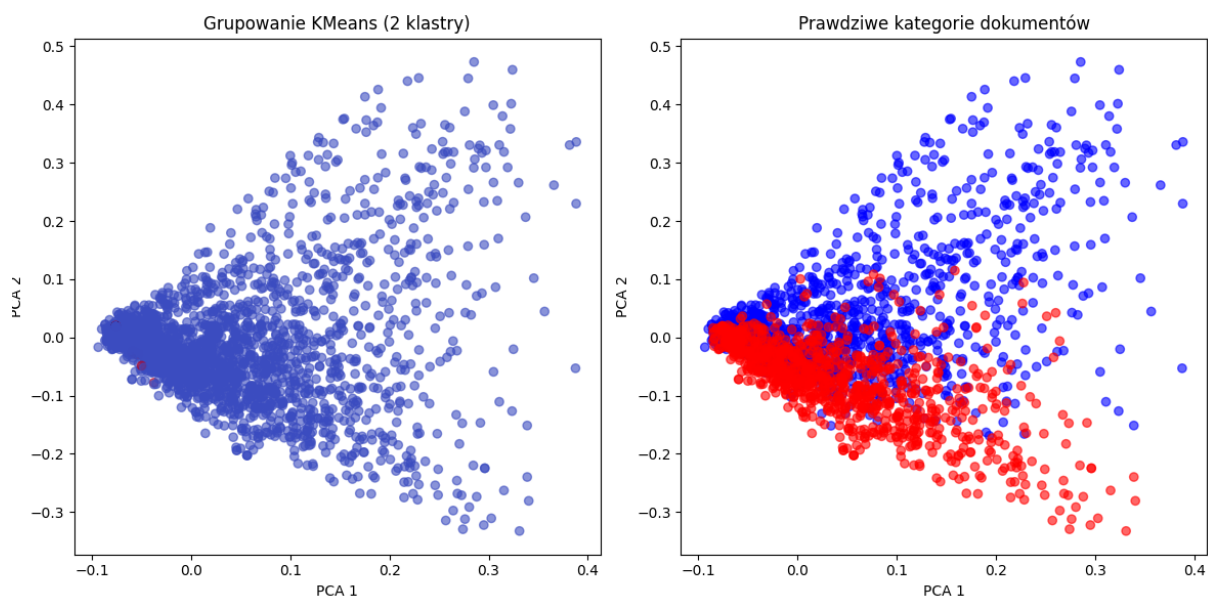
lda = LDA(n_components=2, random_state=42)
lda_topics = lda.fit_transform(X_tfidf)

#Najważniejsze słowa dla tematów
feature_names = vectorizer.get_feature_names_out()

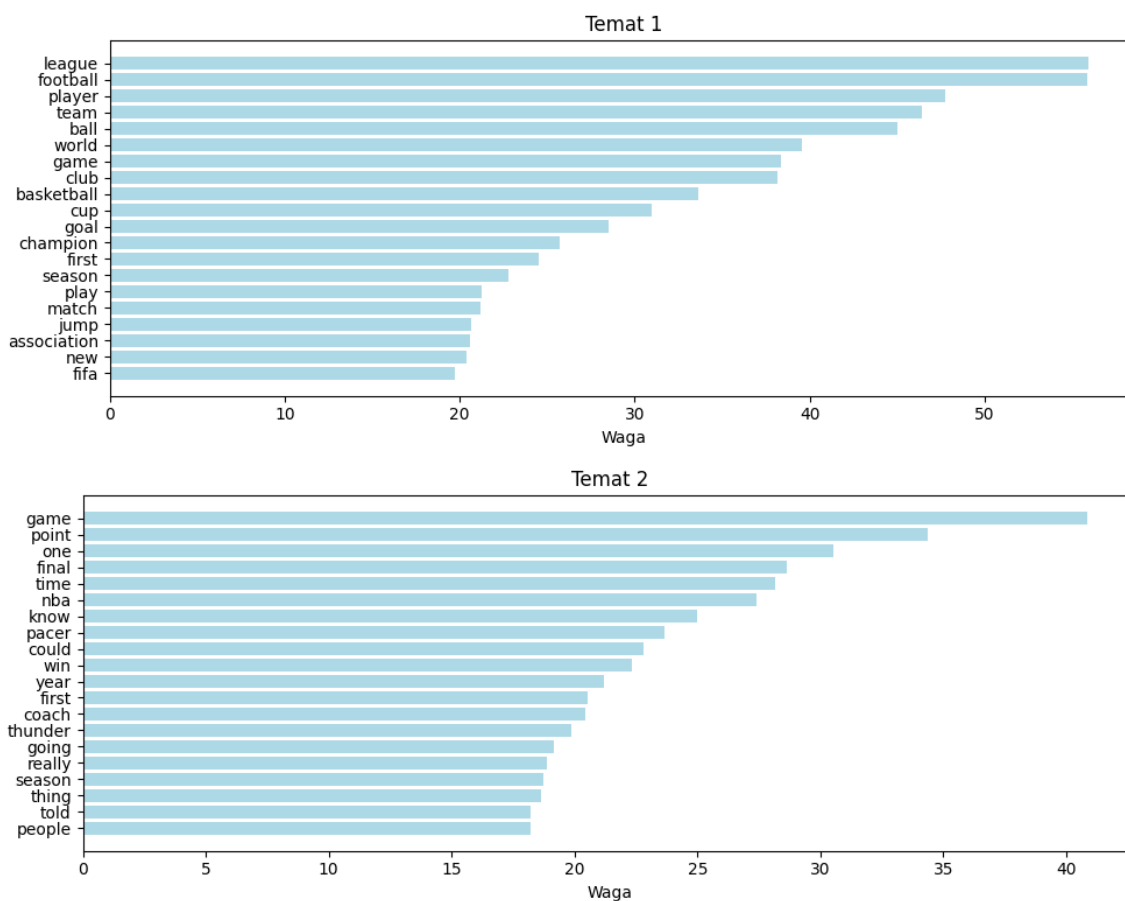
def print_top_words(model, feature_names, n_top_words=20):
    for topic_idx, topic in enumerate(model.components_):
        print(f"\nTemat {topic_idx+1}:")
        top_words = [feature_names[i] for i in topic.argsort()[::-n_top_words - 1::-1]]
        print(", ".join(top_words))

print("\nModelowanie tematów (LDA):")
print_top_words(lda, feature_names)
```

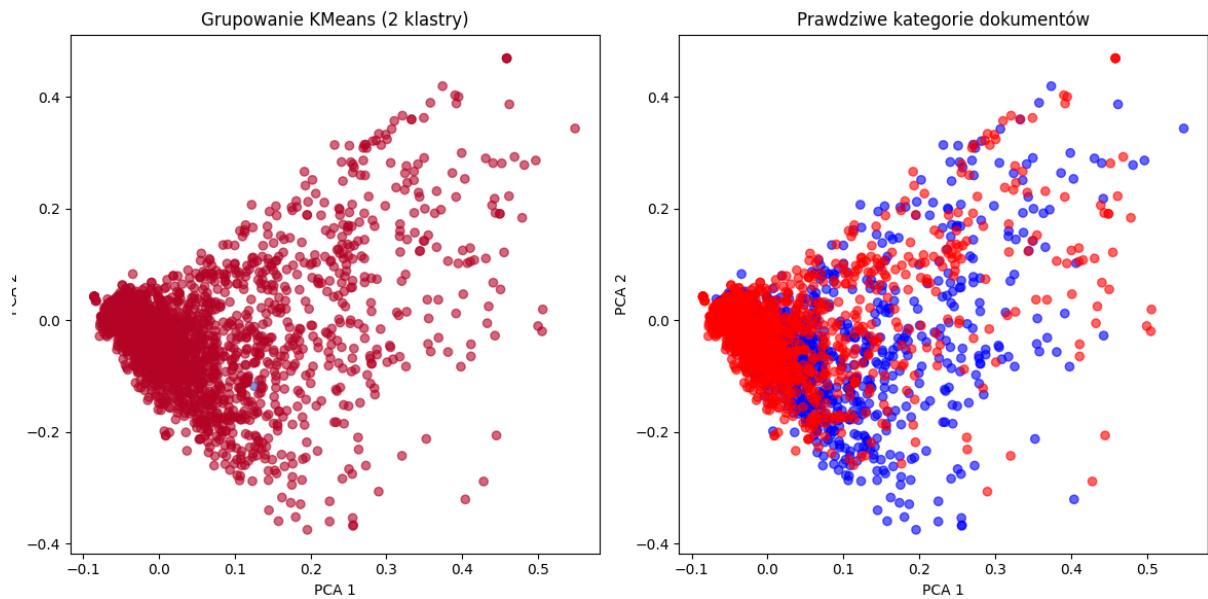
Rysunek 8. Proces grupowania, generowania tematów z LDA i funkcja generowania najważniejszych słów dla danych tematów.



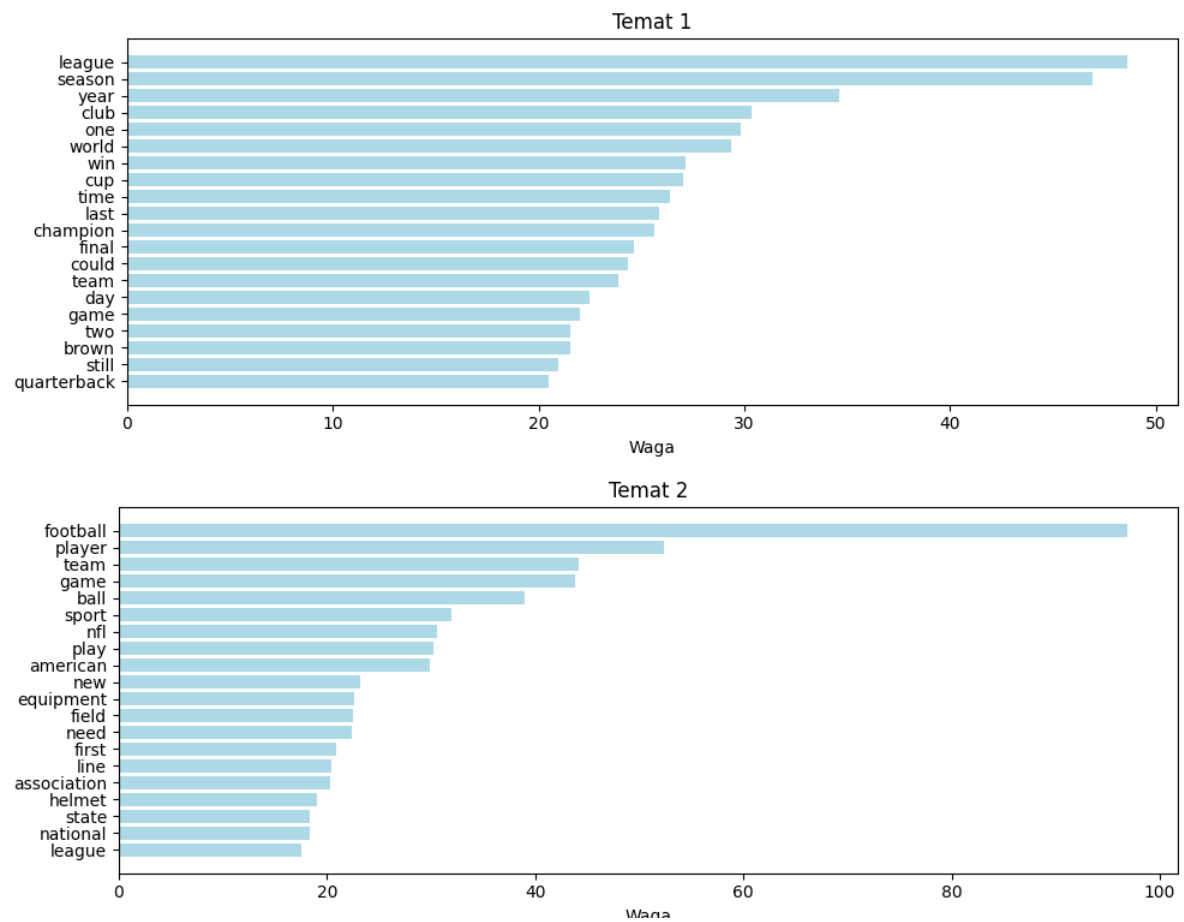
Rysunek 9. Grupowanie artykułów o piłce nożnej (niebieskie) i koszykówce (czerwone).



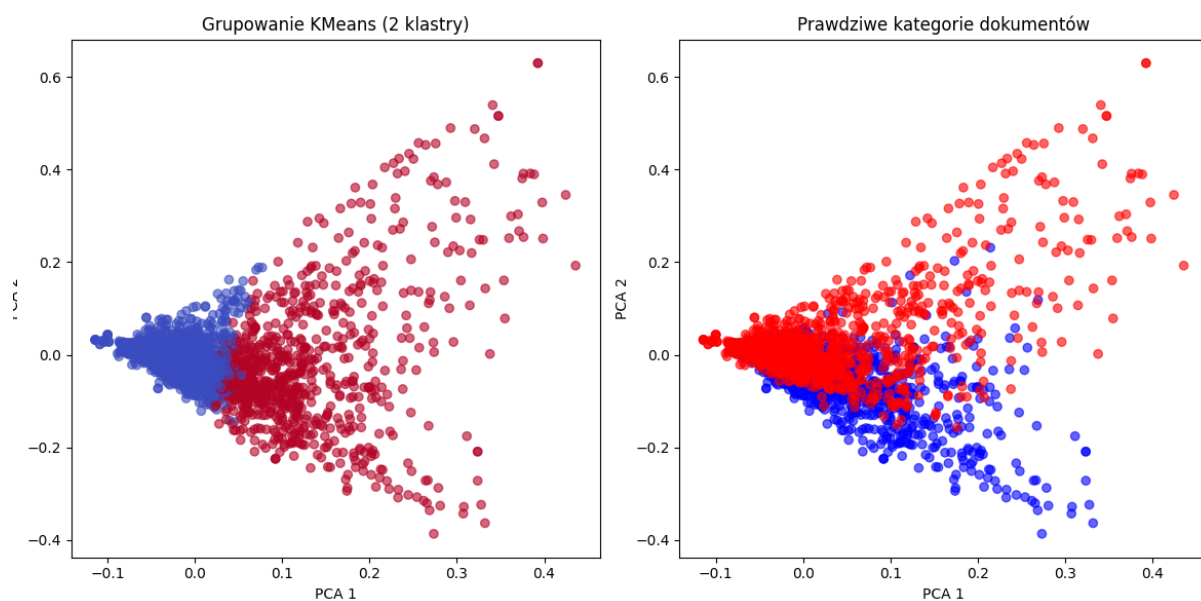
Rysunek 10. Najważniejsze słowa dla dwóch tematów w artykułach o piłce nożnej i koszykówce.



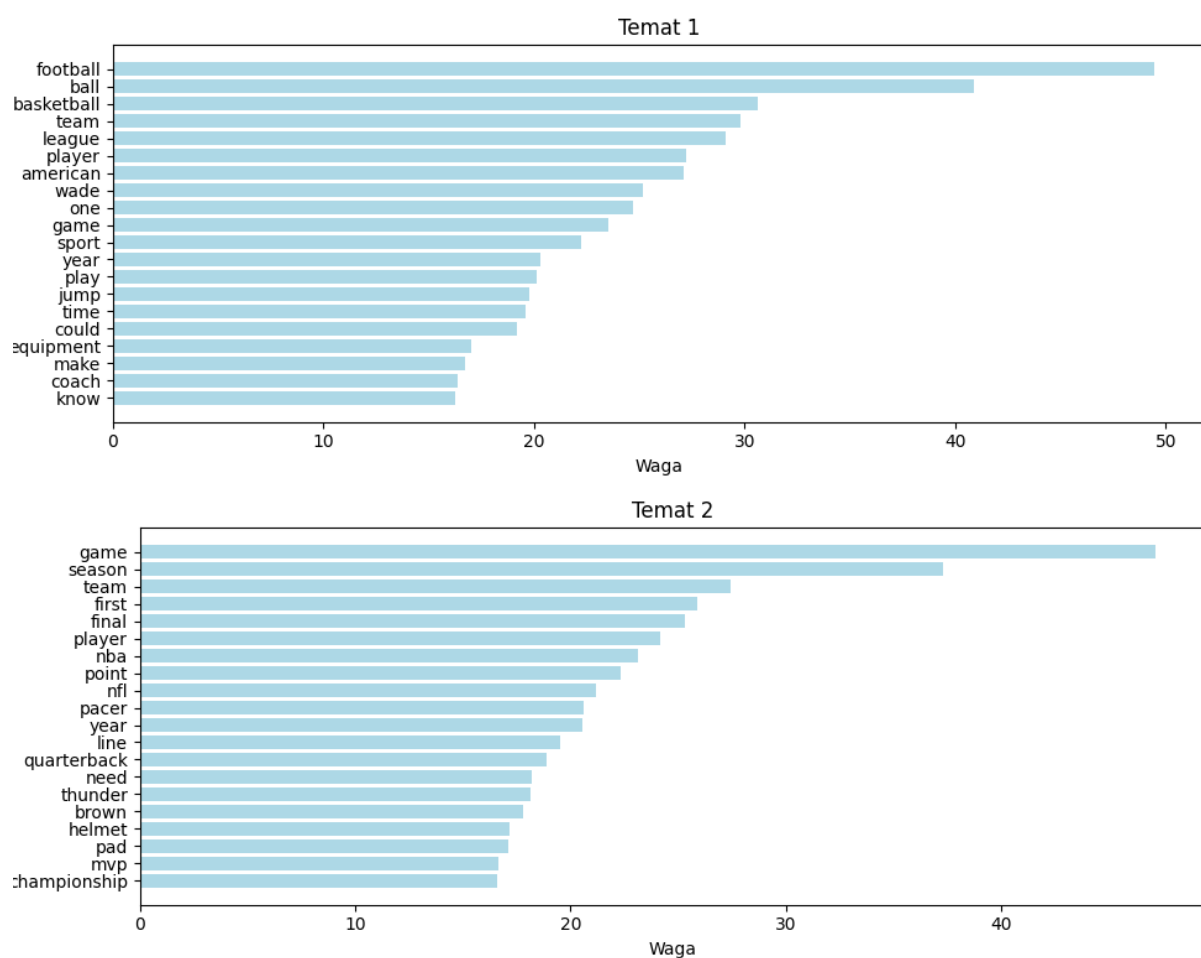
Rysunek 11. Grupowanie artykułów o piłce nożnej (niebieskie) i futbolu amerykańskim (czerwone).



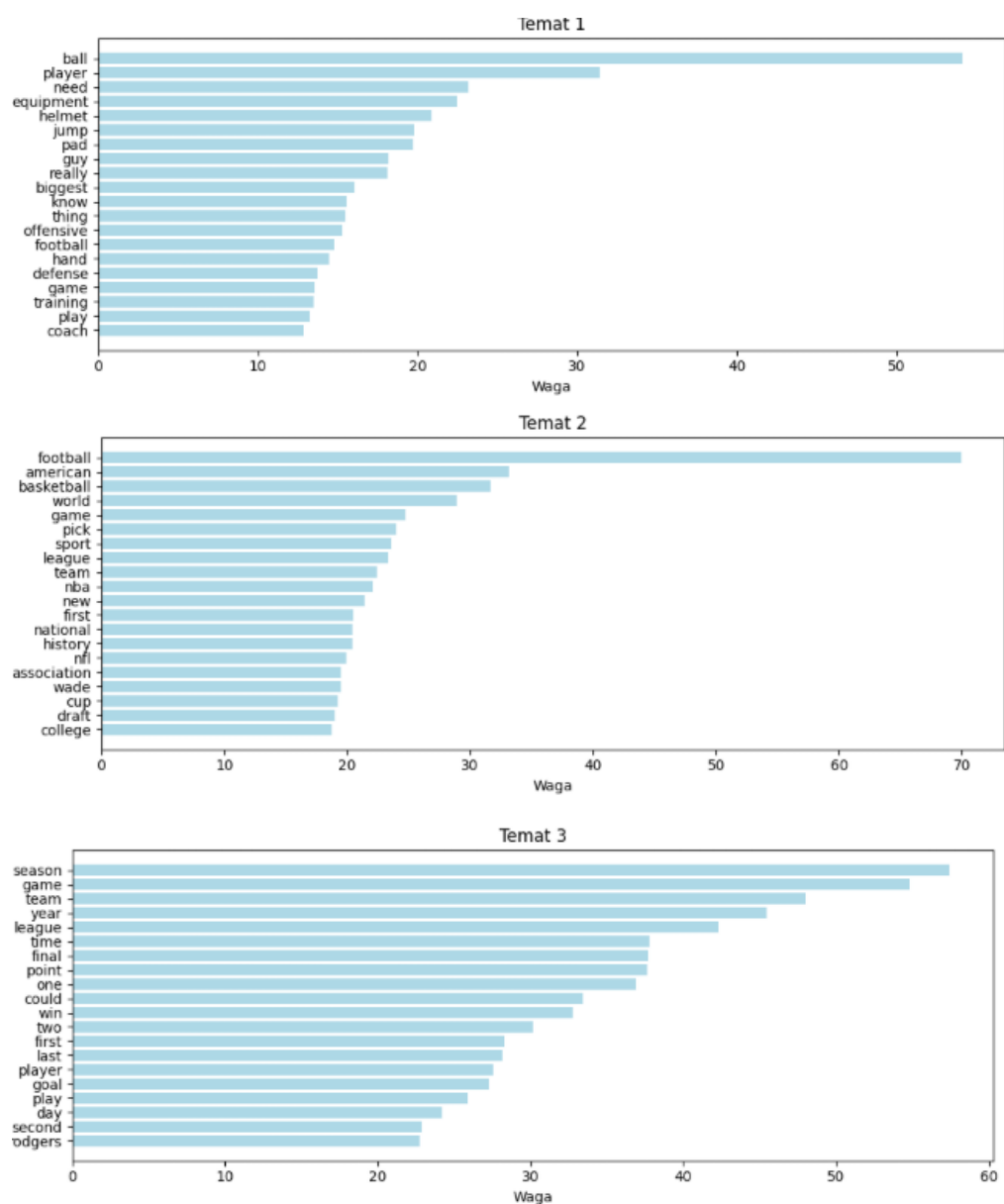
Rysunek 12. Najważniejsze słowa dla dwóch tematów w artykułach o piłce nożnej i futbolu amerykańskim.



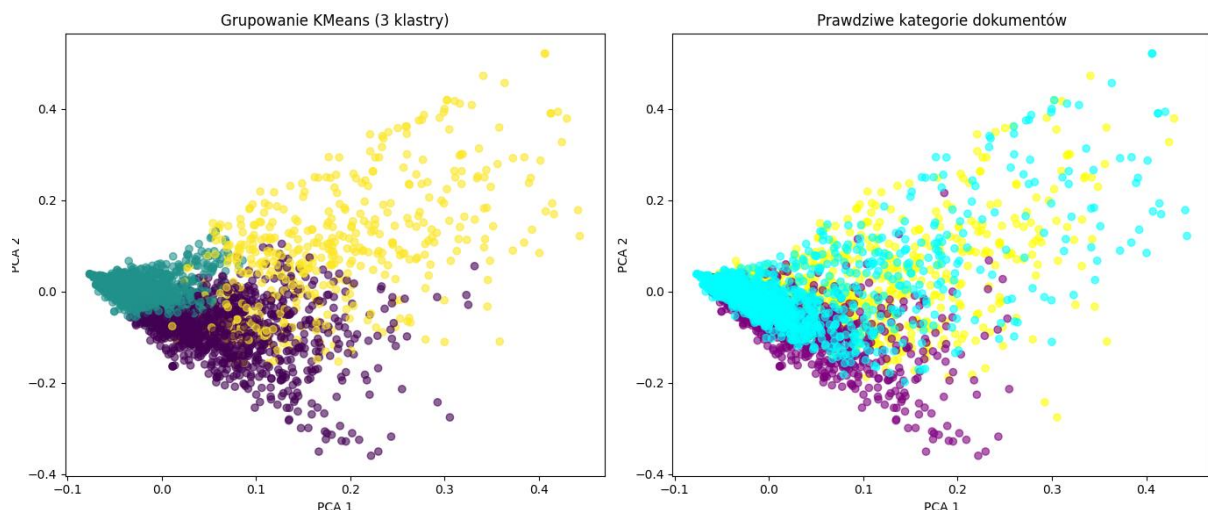
Rysunek 13. Grupowanie artykułów o koszykówce (niebieskie) i futbolu amerykańskim (czerwone).



Rysunek 14. Najważniejsze słowa dla dwóch tematów w artykułach o koszykówce i futbolu amerykańskim.



Rysunek 15. Najważniejsze słowa dla trzech tematów w artykułach o piłce nożnej, koszykówce i futbolu amerykańskim.



Rysunek 16. Grupowanie artykułów o koszykówce (fioletowe), futbolu amerykańskim (cyjanowe) i piłce nożnej (żółte).

Wyniki grupowania

Niestety dla wszystkich następujących grupowań, algorytm nie odróżnia skutecznie różnic pomiędzy dyscyplinami a wizualizacja PCA pokazuje znaczne nakładanie się obu kategorii. Pomieszczenie punktów na wykresach sugeruje, że PCA nie znalazło wyraźnych kierunków różnicujących tekstów- i że owe różnice są zbyt subtelne dla jedynie dwóch grup. Spośród trzech grupowań pomiędzy dwoma sportami, najwięcej różnic wykryło porównanie koszykówki i futbolu amerykańskiego- jednak nachodzenie na siebie punktów wciąż świadczy o podobieństwie słów. Przyczyną błędu w grupowaniu może być częste używanie tej samej terminologii i poruszania tych samych tematów (np. awans drużyny, relacja meczu, doświadczenia sportowców). Zaważyła też sama ilość badanego tekstu- w trakcie testowania na liczbie znaków poniżej 200 000 dla jednego pliku, grupowanie przynosiło bardziej zadawalające efekty niż dla końcowych wersji. Grupowanie wszystkich połączonych dokumentów przyniosło o niewiele więcej lepsze rezultaty.

Wyniki modeli wątków

W przypadku analizy dwóch zbiorów, algorytm LDA zidentyfikował tematy, które odzwierciedlały rzeczywiste różnice między dyscyplinami. Najważniejsze słowa przypisane do każdego z tematów były zgodne z właściwą terminologią. Najlepiej wątki zostały rozdzielone na 2 dla dwóch pierwszych analiz (piłka nożna i koszykówka oraz piłka nożna i futbol amerykański). Niestety w zbiorze najlepiej określających słów dla danego tematu, znalazły się też słowa niepasujące. Jednak biorąc pod uwagę wielkość korpusu i podobieństwo w słownictwie, wizualna inspekcja słów i histogramy tematyczne potwierdzały, że LDA wykrywa wzorce w danych- przynajmniej dla dwóch kategorii.

W przypadku próby modelowania tematów dla trzech dokumentów, efekty były jednak znacznie mniej satysfakcjonujące. Tematy nie rozdzielały się tak wyraźnie, a najważniejsze słowa dla każdego z tematów były często zbliżone lub zbyt ogólne, by dało się przypisać je

jednoznacznie do konkretnej dyscypliny. Powodem takiego wyniku były przypuszczalnie te same czynniki które nie pozwoliły na poprawne pogrupowanie- podobieństwo słownictwa, nadmiar dokumentów i zbyt mała liczba tematów.

4. Klasyfikacja

Następnym celem było przetestowanie różnych klasyfikatorów, aby sprawdzić jak radzą sobie z rozpoznawaniem kategorii tekstów sportowych na podstawie stworzonego korpusu. Badanie przeprowadzono dla czterech metod reprezentacji tekstów:

- Binary- zliczanie słów,
- Logarithmic- zliczanie słów z przekształceniem logarytmicznym,
- TF_IDF- zliczanie uwzględniające częstość słowa w dokumencie i korpusie,
- Word Embeddings- wektory GloVE.

W projekcie wykorzystano gotowy model "GloVE" (Global Vectors for Word Representation) udostępniony przez bibliotekę gensim. Zawiera 100-wymiarowe wektory wytrenowane na danych z wikipedii i korpusu Gigaword. Dodano także własne uczenie z "Word2Vec" na podstawie naszego korpusu- wyniki jednak nie miały wystarczającego znaczenia aby zastąpić tą metodą pobrany wcześniej model.

Dla każdej metody wykorzystano klasyfikatory:

- Random Forest
- SVM
- Gaussian Naive Bayes
- k_NN (tylko z Word Embeddings)

```

classifiers = {
    "Gaussian NB": GaussianNB(),
    "SVM": SVC(kernel='linear', random_state=42, class_weight='balanced'),
    "Random Forest": RandomForestClassifier(class_weight='balanced', random_state=42)
}

vectorization_methods = {
    "Binary": CountVectorizer(binary=True, max_features=2000),
    "Logarithmic": (CountVectorizer(max_features=2000), FunctionTransformer(log_transform)),
    "TF-IDF": TfidfVectorizer(max_features=2000, ngram_range=(1, 2))
}

class WordEmbeddingVectorizer(BaseEstimator, TransformerMixin):
    def __init__(self, embedding_model):
        self.embedding_model = embedding_model
        self.dim = embedding_model.vector_size

    def fit(self, X, y=None):
        return self

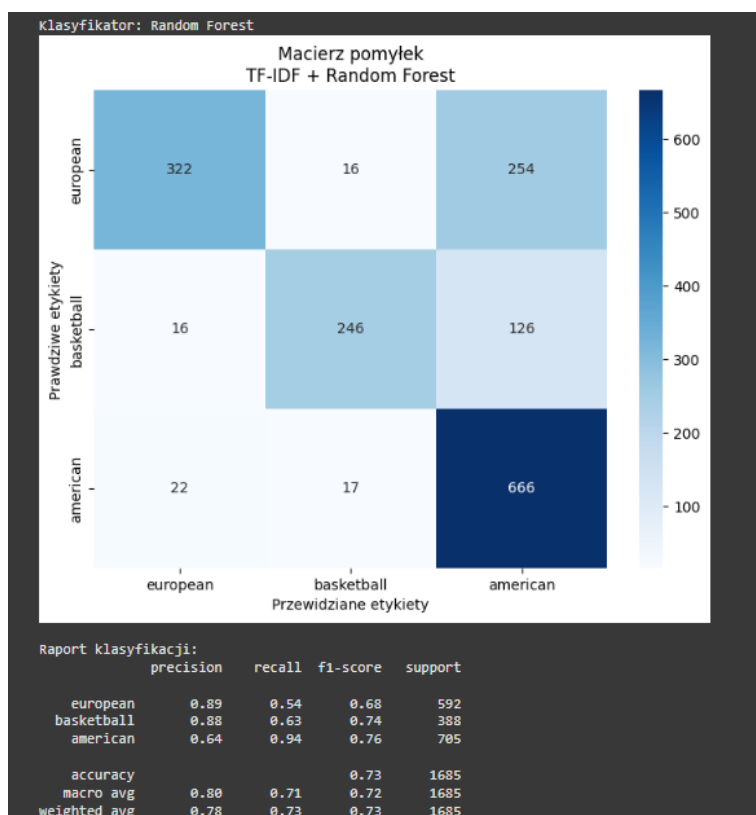
    def transform(self, X):
        return np.array([
            np.mean([self.embedding_model[word] for word in doc.split()
                     if word in self.embedding_model] or [np.zeros(self.dim)], axis=0)
            for doc in X])

try:
    glove_vectors = api.load("glove-wiki-gigaword-100") #pobieranie modelu word embeddings GloVe
except:
    sentences = [doc.split() for doc in texts]
    glove_vectors = Word2Vec(sentences, vector_size=100, window=5, min_count=1).wv

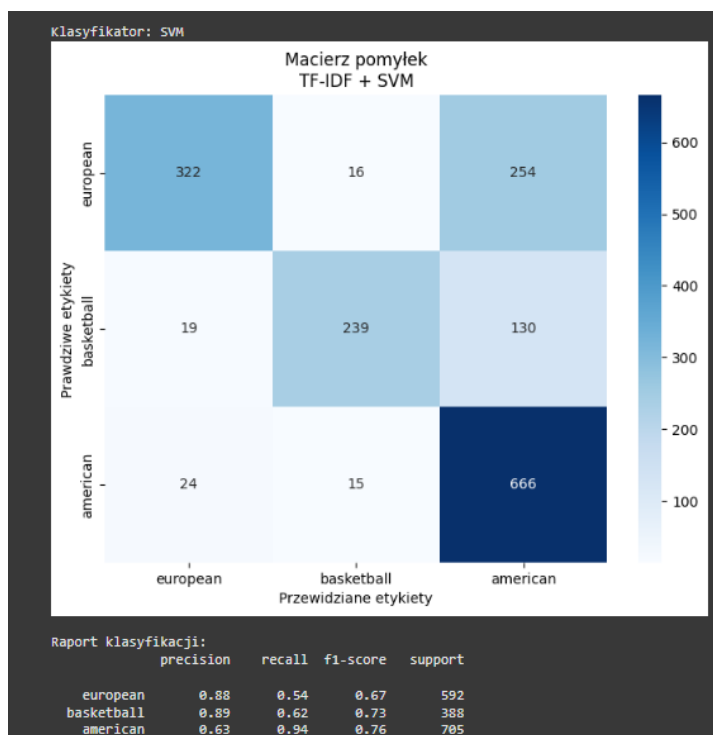
vectorization_methods["Word Embeddings"] = WordEmbeddingVectorizer(glove_vectors)

```

Rysunek 17. Przygotowanie modeli klasyfikacji, wag termów i pobranie pretrenowanego modelu GloVe.



Rysunek 18. Wynik klasyfikacji Random Forest z TF-IDF.



Rysunek 19. Wynik klasyfikacji TF-IDF z SVM.

Wyniki analizy

Podczas wcześniejszych etapach pracy zauważono, że modele lepiej radziły sobie, gdy dokumenty były krótsze, czyli zawierały mniej słów – jak we wcześniejszych etapach projektu. Wówczas klasyfikacja była prostsza i mniej podatna na szum informacyjny, co wynika prawdopodobnie z mniejszego nakładania się słownictwa między klasami.

W przypadku pełnych artykułów (zestawy końcowe), wyniki nieco się pogorszyły — co sugeruje, że nadmiar kontekstu niekoniecznie wspiera rozróżnienie pomiędzy artykułami sortowymi- zwłaszcza dla piłki nożnej i futbolu amerykańskiego.

Klasa dotycząca futbolu amerykańskiego dominowała w liczebności we wszystkich tekstach (pomimo początkowej porównywalnej liczbie słów przed czyszczeniem) co przełożyło się na wyższą skuteczność dla tej kategorii, pogarszając tym samym klasyfikację dla pozostałych sportów. K

Najlepszy model

Spośród wszystkich przetestowanych kombinacji, najlepszy wynik osiągnął model Random Forest z metodą wagowania TF-IDF gdzie celność (Accuracy) wyniosła 73% a F1-score dla danych tekstów pokazał najwyższe wyniki.

Ten model uzyskał dobrą równowagę między klasami i najmniej mylił dokumenty piłki nożnej z futbolem amerykańskim, co było szczególnie trudnym przypadkiem. Zbliżone wyniki osiągnął model SVM z metodą wagowania TF-IDF. Word Embeddings, mimo przewagi semantycznej, nie przewyższyły klasycznych metod wektorowych- co

prawdopodobnie było wynikiem podobnego słownictwa między klasami. Klasa dotycząca koszykówki była tutaj najbardziej neutralna a błędy w kwalifikacji piłki nożnej i futbolu amerykańskiego były mniejsze niż w pozostałych metodach.

5. Polaryzacja nastroju

Ostatnim etapem analizy było zbadanie emocji dominujących w artykułach sportowych dla wszystkich trzech dyscyplin oraz zbadanie polaryzacji nastroju dla danych tekstów. W celu przeprowadzenia tego testu, wykorzystano zasób leksykalny "NRC Emotion Lexicon", zawierający powiązanie słów z dziesięcioma emocjami oraz samymi nastrojami "positive" i "negative".

Analiza emocji nastąpiła poprzez zliczenie częstości występowania poszczególnych emocji w oparciu o tokeny słów z dokumentów tekstowych.

Samą polaryzację nastroju obliczono osobno zliczając słowa pozytywne i negatywne. Liczbę słów pozytywnych podzielono przez sumę słów pozytywnych i negatywnych, mnożąc wyrażenie przez 100.

```
def analyze_emotions(tokenized_docs, nrc_dict):
    emotion_counter = Counter()
    for doc in tokenized_docs:
        for word in doc:
            if word in nrc_dict:
                for emotion in nrc_dict[word]:
                    emotion_counter[emotion] += 1
    return emotion_counter

emocje_basketball = analyze_emotions(basketball_clean, nrc)
emocje_european = analyze_emotions(european_clean, nrc)
emocje_american = analyze_emotions(american_clean, nrc)

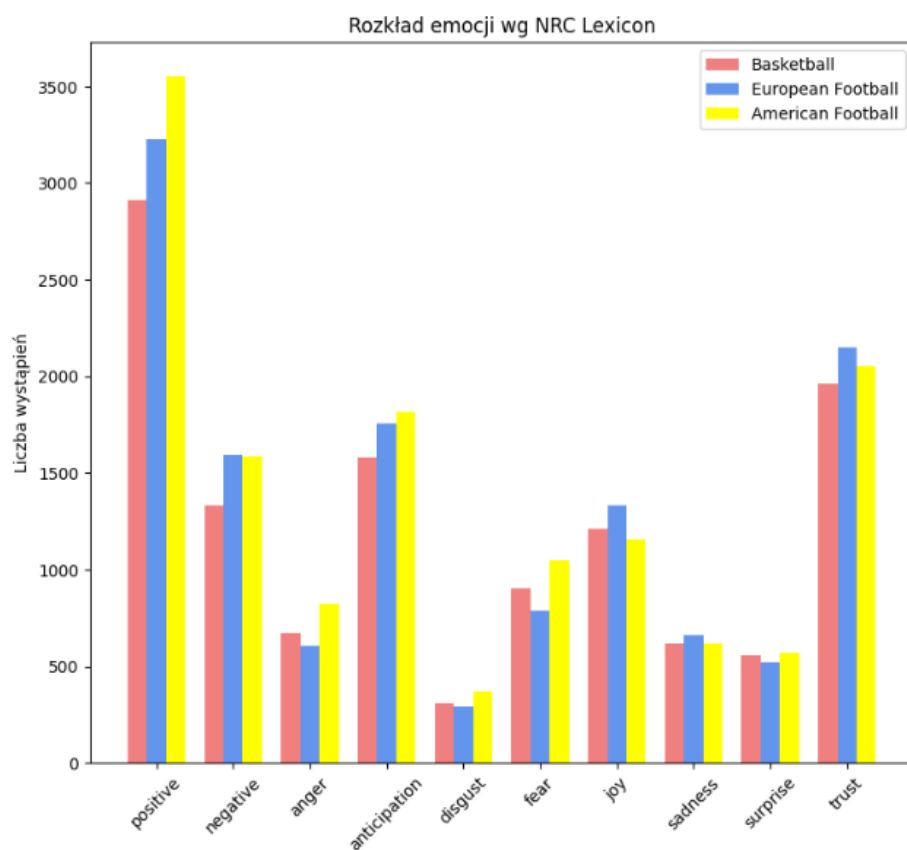
def calculate_polarity(tokenized_docs, nrc_dict):
    positive = negative = 0
    for doc in tokenized_docs:
        for word in doc:
            if word in nrc_dict:
                if 'positive' in nrc_dict[word]:
                    positive += 1
                if 'negative' in nrc_dict[word]:
                    negative += 1
    return positive, negative

pos_bs, neg_bs = calculate_polarity(basketball_clean, nrc)
pos_eu, neg_eu = calculate_polarity(european_clean, nrc)
pos_am, neg_am = calculate_polarity(american_clean, nrc)

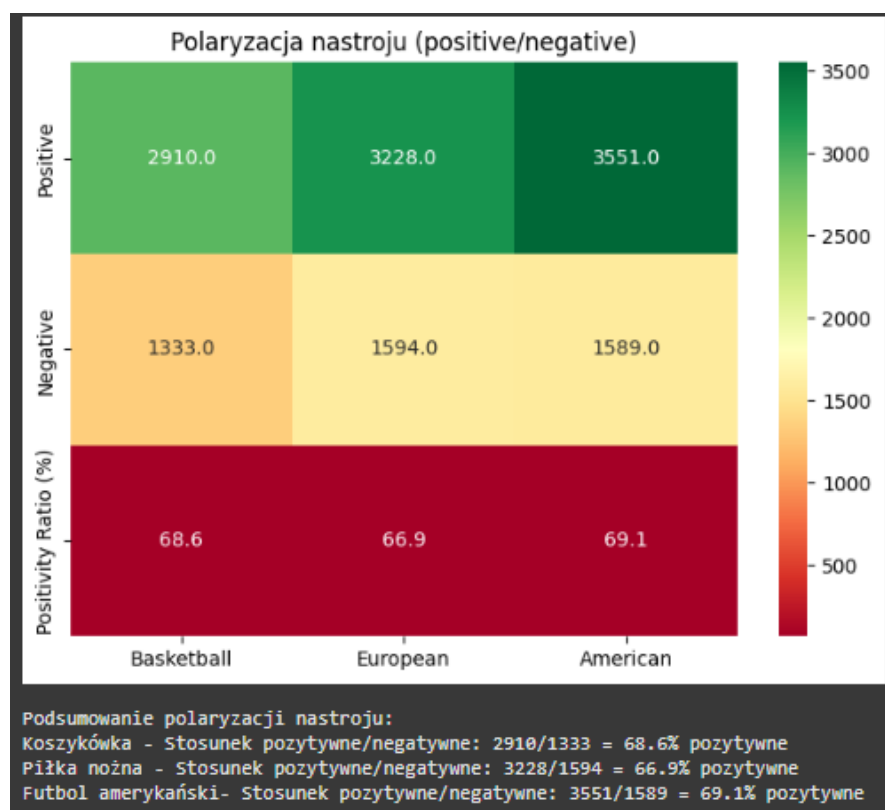
plt.figure(figsize=(20, 8))

emocje = ['positive', 'negative', 'anger', 'anticipation', 'disgust',
          'fear', 'joy', 'sadness', 'surprise', 'trust']
```

Rysunek 20. Funkcje analizy emocji i obliczenia wskaźnika polaryzacji.



Rysunek 21. Histogram rozkładu emocji we wszystkich artykułach sportowych.



Rysunek 22. Podsumowanie polaryzacji nastroju dla wszystkich artykułów.

Wyniki polaryzacji nastroju

Na podstawie testu polaryzacji nastroju, zauważyć można bardzo podobny rozkład nastroju dla wszystkich trzech zbiorów artykułów sportowych. Wszystkie trzy kategorie mają przewagę słów pozytywnych. Różnice głównie są zauważalne tylko przez liczbę termów w korpusach. Sam wskaźnik nastroju również jest zbliżony dla wszystkich trzech tekstów.